

수업 08 · 전체 과정 13/20일차

전자회로·ESP32

2026-10-23 (금) · 10:00~17:00

50분 수업 · 10분 휴식 · 중식 11:50~13:00

전압, 전류, 저항을 배우고 ESP32의 입출력을 살펴봅니다. 회로를 종이로 검토하고 가상 센서 기록을 읽어 봅니다.

교안 읽기	직접 해보기	결과 확인하기
개념과 완성 예제를 먼저 봅니다.	필수 활동을 열어 작성·수정합니다.	예상과 실제를 비교하고 설명합니다.

처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

웹 강의실 열기 · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

이 교안의 순서

오늘 준비할 것	3
오늘의 6교시	4
1교시 · 전기 기초와 계산	5
2교시 · 부품·보드·입출력	10
3교시 · 변환·스키마·모의입력	15
4교시 · 경계·흔들림·기록 보존	20
5교시 · 펌웨어·통신·배선 검토	26
6교시 · 새 요구·종합 검수·인수인계	31
용어와 코드 기호 찾아보기	37
오늘 배운 내용 확인하기	39
파일과 다음 수업	42

오늘 준비할 것

2026-10-23 · 처음 시작

Python 표준 라이브러리로 가상 센서 파일을 읽습니다. 실제 연결·통전·시리얼 포트 열기는 하지 않습니다. .ino 파일은 보드용 코드를 읽는 자료이며, 컴파일하거나 업로드한 결과는 아닙니다.

파일과 실행

examples/course/data는 읽을 입력, code는 완성 시연 코드, work는 나의 작성·수정 파일입니다. ZIP을 풀고 폴더 구조를 유지합니다. 터미널은 명령을 실행하는 창이며, 에디터는 파일 내용을 쓰고 저장하는 창입니다. 저장만으로 코드가 실행되지는 않습니다.

1. 용어에서 낯선 말을 확인합니다.
2. 교안의 개념과 예를 읽습니다.
3. 활동지의 A01부터 입력을 열고 work 양식에 기록합니다.
4. 코드 실행은 code/line-guide.md의 명령과 조건을 따릅니다.

확인할 것: 전압·전류·저항의 단위를 구별하고, 0·null·문자열이 서로 다른 상태인 이유를 설명해 보세요. 가상 입력에서 확인한 결과와 실제 장비에서 확인할 사항도 구별합니다.

개인 코드 실행에는 Python 3를 사용할 수 있는 PC가 필요합니다. 실제 장비를 사용할 때는 정확한 보드·센서·저항·LED 모델을 먼저 확인하세요. 전원·핀·배선을 추측하여 통전하지 않습니다.

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

단위

질문: 3.3V와 3.3mA를 같은 크기로 비교하나요?

풀이: 전압은 밀어주는 정도, 전류는 흐르는 양이며 다른 물리량입니다. 숫자만 같다고 같은 조건이 아닙니다.

회로 경로

질문: LED와 저항을 놓기만 하면 전류가 흐르나요?

풀이: 전원에서 부품을 거쳐 돌아오는 연결 경로가 필요합니다. 회로 검토와 실제 전원 인가를 구분합니다.

판정

질문: 모의 입력에서 명령이 나왔다면 실물 LED도 켜졌나요?

풀이: 코드의 판단 결과를 확인한 것입니다. 핀 연결·전압·보드 환경을 확인한 실물 검증은 별도입니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

오늘의 6교시

교시·시간	학습 초점	필수 활동
1교시 10:00~10:50	전기의 세 값과 닫힌 경로	A01, A02, A03
2교시 11:00~11:50	부품·보드·신호	A04, A05, A06
3교시 13:00~13:50	원시값에서 기록까지	A07, A08, A09
4교시 14:00~14:50	경계·흔들림·원시값	A10, A11, A12
5교시 15:00~15:50	펌웨어·통신·배선 검토	A13, A14, A15
6교시 16:00~16:50	새 센서 요구와 모의 검수	A16, A17, A18

설명·시연·함께 수행·개별 수행·폴이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

값을 읽는 코드가 실제 세상과 만나려면 전기 신호의 약속이 필요합니다. 오늘은 회로를 종이에서 읽고, 센서 대신 제공 파일을 실행하여 신호·값·판단의 관계를 확인합니다. 실제 연결과 통전은 하지 않습니다.

수업의 출발점

물리량의 단위→회로의 경로와 기준→보드와 신호→디지털 기록→검수 순서로 학습합니다. 하드웨어 모델이 정해지지 않았으므로 모든 구체 핀 번호와 배선은 보류하며, 모의 값으로 확인할 수 있는 범위만 실제 실행합니다.

읽는 방법

개념의 뜻을 먼저 읽고 가상 입력에서 근거를 찾습니다. 최신 운영은 50분 수업 × 6교시이며 순수 수업 300분, 중식 11:50~13:00입니다. 기존 활동은 한 교시의 개념 보충·시연·필수 적용·폴이를 위한 자료 묶음입니다. 복습·도전 문제는 진도에 따라 선택합니다. 모든 활동을 50분 안에 모두 완수한다고 가정하지 않습니다. 실제 학생 수행시간은 아직 측정하지 않았습니다.

1교시 · 전기 기초와 계산

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 폴이·확인 7분

이번 교시에서 확인할 것. 전기의 세 값과 닫힌 경로

먼저 할 활동: A01, A02, A03

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A01 · 전압·전류·저항을 구분

1교시 필수 개념 보충 · 전류가 돌아오는 길과 단위 계산

실무 상황과 원리

실습 담당자가 3.3V 전원과 330Ω 저항을 종이에 연결합니다. “3.3이 흐른다”라는 말로는 무엇을 측정하는지 알 수 없습니다. 전압은 두 점 사이의 차이이고 단위는 V입니다. 전류는 시간당 이동하는 전하량이며 A, 저항은 전압과 전류의 관계를 나타내며 Ω입니다. 물의 압력 차이와 유량 비유는 역할을 기억하는 출발점으로만 사용합니다.

전류의 통상적인 방향을 전원 + → 저항 → 전원 -로 표시합니다. 돌아오는 길까지 이어져야 닫힌 회로입니다. 중간이 끊어진 이상적인 회로에는 지속적인 전류가 흐르지 않습니다. GND는 이 회로에서 전압 비교의 기준으로 정한 지점이며 언제나 건물의 보호 접지라는 뜻은 아닙니다. 저항 없이 전원 +와 -를 잇는 것은 부하를 통과하는 정상 경로와 다릅니다.

완성 계산의 입력은 저항 양단 전압 3.3V와 저항 330Ω입니다. 옴성 저항의 관계 $V=IR$ 에서 $I=V/R$ 로 바꿉니다. $3.3 \div 330 = 0.010A$ 이고, $1A = 1000mA$ 이므로 10mA입니다. 저항을 660Ω으로 바꾸면 $0.005A = 5mA$ 입니다. 나눗셈 결과에 바로 mA를 붙이면 1000배의 단위 오류가 됩니다. 계산은 이상적인 저항의 관계를 확인한 것이며 부품 발열, 허용 전

류, 보드 출력 능력을 승인한 결과는 아닙니다. 실제 연결 전에는 부품과 보드 사양이 필요합니다. 모의 계산의 실행 여부와 실물 측정 여부를 기록 칸에서 나눕니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 저항 양단 5V와 1000Ω이라는 새 종이 조건에서 A와 mA를 각각 계산하고 전류가 돌아오는 경로를 화살표로 그리세요. 가운데 선을 지웠을 때의 이상적 전류도 쓰세요.

정답과 오답의 이유

0.005A=5mA이며 + → 저항 → -입니다. 선이 끊어진 이상적 경로는 0A입니다. 5V를 ESP32 신호 입력에 연결해도 된다는 뜻은 아닙니다. 단위 계산과 연결 허용 판단은 별도 문제입니다.

확인 질문과 답

- 1. 전압은 몇 점 사이인가요? → 두 점 사이입니다.
- 2. 0.002A는? → 2mA입니다.
- 3. GND는 버리는 구멍인가요? → 전압 기준과 반환 경로의 역할을 읽어야 합니다.

전압은 두 점 사이의 전기적 에너지 차이를 전하량으로 나눈 값이고 단위는 V입니다. 전류는 시간당 흐르는 전하량이며 A, 저항은 전류 흐름을 제한하는 관계를 나타내며 Ω로 씁니다.

단위가 다른 수를 같은 값처럼 연결하면 회로 계산과 부품 선택을 잘못합니다. 수도관의 압력차·유량·흐름 저항 비유는 역할을 기억하는 데만 씁니다.

입력에서 살펴볼 예

전압3.3V 전류0.010A=10mA 저항330Ω

전압은 두 점 사이에서 말합니다. 전류는 흐름이며, 저항은 그 흐름과 전압의 관계입니다.

구분해야 할 반례

전압 자체가 회로를 흐른다고 말하지 않습니다. 물 비유로 전기 현상의 모든 성질을 설명할 수 없습니다.

스스로 설명할 질문: 3.3V와10mA를 서로 바꿔 쓸 수 없는 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A01 · 따라 하기와 작성칸

A01 · 전압·전류·저항을 구분

목적 단위가 다른 수를 같은 값처럼 연결하면 회로 계산과 부품 선택을 잘못합니다. 수도관의 압력차·유량·흐름 저항 비유는 역할을 기억하는 데만 씁니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A01_input.md](#)

작업: [A01_worksheet.md](#)

단계

1. 세 값을 이름과 단위에 연결합니다.
2. A를mA로 바꿉니다.
3. 전압이 어느 두 점 사이인지 질문을 작성합니다.

예상 결과 입력 근거를 표시하고 물리량과 단위에 판단을 남깁니다.

확인 방법 3.3V와10mA를 서로 바꿔 쓸 수 없는 이유는 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A02 · 닫힌 경로와 GND

회로는 전류가 돌아오는 경로를 포함한 연결입니다. GND는 회로에서 전압의 기준으로 정한 지점입니다. 길을 돌아 출발점에 오는 비유로 닫힌 경로를 읽습니다.

신호의 높고 낮음은 기준에 대한 값입니다. 연결 이름뿐 아니라 돌아오는 경로와 기준 공유 조건을 이해해야 합니다.

입력에서 살펴볼 예

종이 회로: 전원+ → 저항 → 전원- 중간 끊김: 열린 경로 기준: 전원-를0V로 정함

완전히 이어진 경로와 끊긴 경로를 구분합니다. GND는 신호를 비교할 기준입니다.

구분해야 할 반례

모든 GND가 건물의 보호접지와 같지는 않습니다. 전원+와GND를 바로 잇는 것은 정상 부하 경로가 아니라 단락 위험입니다.

스스로 설명할 질문: GND를 모두 안전한 버리는 구멍으로 생각하면 왜 틀리나요?

활동 연결: [입력](#) · [기록 양식](#)

A02 · 따라 하기와 작성칸

A02 · 닫힌 경로와 GND

목적 신호의 높고 낮음은 기준에 대한 값입니다. 연결 이름뿐 아니라 돌아오는 경로와 기준 공유 조건을 이해해야 합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A02_input.md](#)

작업: [A02_worksheet.md](#)

단계

1. 종이에서 돌아오는 경로를 표시합니다.
2. 열린 곳과 기준점을 표시합니다.
3. 단락 예를 찾아 연결 금지 이유를 적습니다.

예상 결과 입력 근거를 표시하고 경로 표시에 판단을 남깁니다.

확인 방법 GND를 모두 안전한 버리는 구멍으로 생각하면 왜 틀리나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A03 · 옴의 법칙을 계산으로 확인

일정 조건에서 옴성 저항은 $V=IR$ 관계를 따릅니다. 전류를 구할 때 $I=V/R$ 로 바꾸고 단위를 맞춥니다. LED 같은 비선형 부품 전체에 고정 저항처럼 적용하지 않습니다.

전압이 같을 때 저항을 두 배로 하면 이상적인 계산 전류가 절반이라는 관계를 확인합니다. 계산 결과와 부품의 실제 허용 정격은 별개입니다.

입력에서 살펴볼 예

저항 양단 3.3V, 저항 330Ω $I=3.3/330=0.010A$ 660Ω로 변경

330Ω 계산은 10mA입니다. 660Ω 계산은 5mA입니다. 부품 정격과 발열은 따로 확인합니다.

구분해야 할 반례

이 계산만으로 ESP32핀에서 10mA를 흘려도 된다고 승인할 수 없습니다. 보드·칩·핀·부품의 문서가 더 필요합니다.

스스로 설명할 질문: 계산 전류와 실제 연결 허용 여부는 왜 다른가요?

활동 연결: [입력](#) · [기록 양식](#)

01_ohm.py 전체

```
voltage_v = 3.3
resistance_ohm = 330
if resistance_ohm <= 0:
    raise ValueError("저항은 양수여야 합니다")
current_a = voltage_v / resistance_ohm
current_ma = current_a * 1000
print("current_mA:", round(current_ma, 2))
```

실행 조건: Python 3 표준 라이브러리만 사용합니다. 장치·포트·전원을 연결하지 않습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/01_ohm.py`

결과:

```
current_mA: 10.0
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 저항을 660으로 바꾸면 5.0mA입니다. 전압은 3.3V를 유지합니다.

오류 복구: 저항0은 단락 모사 결과가 아니라 입력 오류입니다. 실제 단락을 만들어 확인하지 않습니다.

A03 · 따라 하기와 작성칸

A03 · 옴의 법칙을 계산으로 확인

목적 전압이 같을 때 저항을 두 배로 하면 이상적인 계산 전류가 절반이라는 관계를 확인합니다. 계산 결과와 부품의 실제 허용 정격은 별개입니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A03_input.md](#) · [01_ohm.py](#)

작업: [A03_worksheet.md](#)

단계

1. 코드 각 줄을 읽고 결과를 예상합니다.
2. 원본과 660Ω 수정본을 비교합니다.
3. 0Ω 입력을 실제 연결 대신 코드 오류로 확인합니다.

예상 결과 입력 근거를 표시하고 계산 비교에 판단을 남깁니다.

확인 방법 계산 전류와 실제 연결 허용 여부는 왜 다른가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

2교시 · 부품·보드·입출력

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 부품·보드·신호

먼저 할 활동: A04, A05, A06

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A04 · LED와 저항의 역할

2교시 필수 개념 보충 · LED 직렬 저항과 보드 신호의 전압

실무 상황과 원리

알림등을 달고 싶은 담당자는 LED가 밝아지면 성공이라고 생각합니다. 그러나 빛이 보이는 것과 부품이 정격 안에서 동작하는 것은 다릅니다. LED는 방향이 있는 다이오드이며 저항처럼 일정한 V/I 값으로 취급하지 않습니다. 직렬 저항은 같은 가치를 흐르는 전류를 제한합니다.

제작 가정으로 공급 전압 3.3V, LED 전압 강하 2.0V, 직렬 저항 330Ω을 정합니다. 저항에 걸리는 전압은 $3.3 - 2.0 = 1.3V$ 입니다. 전류는 $1.3 \div 330 = 0.00394A$, 약 3.94mA입니다. 저항 양단에 3.3V 전체가 걸린다고 계산한 10mA와 다릅니다. 목표 전류를 가상으로 5mA라 정하면 필요한 이상적 저항은 $1.3 \div 0.005 = 260\Omega$ 입니다. 실제 선택은 LED 사양, 허용 전류, 저항 정격, 오차와 사용 가능한 표준값을 확인하는 후속 작업입니다. 오늘 숫자는 배선 승인값이 아닙니다.

칩은 계산과 입출력을 수행하는 핵심 부품, 모듈은 칩과 주변 회로 묶음, 개발 보드는 연결과 개발을 돕는 전체 기판입니다. 보드에 5V 전원 단자가 있어도 GPIO 입력이 5V를 허용한다는 결론은 나오지 않습니다. GPIO는 프로그램으로 디지털 신호를 읽거나 출력하는 핀이며 HIGH는 정해진 논리 수준을 뜻합니다. 핀 번호, 부팅 관련 기능, 출력 전류 제한은 정확한 ESP32 계열과 보드 문서로 확인해야 합니다. 모델이 미확정이면 구체 배선 대신 확인할 항목을 작성합니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 공급 3.3V, 가정 LED 강하 1.8V, 저항 500Ω에서 전류를 계산하세요. 보드 전원 입력 표시에 5V가 있다는 이유로 센서의 5V 출력을 GPIO에 연결할 수 있는지도 판단하세요.

정답과 오답의 이유

저항 전압은 1.5V, 전류는 3mA입니다. GPIO 허용 전압은 별도 사양으로 확인해야 합니다. LED 강하를 더해 5.1V로 계산하거나 USB 전원과 신호 입력 정격을 같은 것으로 읽으면 오답입니다.

확인 질문과 답

1. 직렬 가지의 전류는? → 같은 가지에서 같습니다.
2. 2.0V는 확정 사양인가요? → 여기서는 계산용 가정입니다.
3. 모델 미확정 시 핀 번호는? → 확정하지 않고 확인 항목으로 남깁니다.

짧은 완성 코드로 추적하기

제공 참고 파일 [code/05_led_calculation.py](#)은 보존합니다. 예제 [course](#) 폴더에서 이 파일을 [work/05_led_calculation.py](#)에 복사하고 학생 수정본은 work에 저장합니다. 아래는 완결된 전체 코드입니다. 파일·폴더 확장자를 확인한 뒤 같은 course 폴더의 터미널에서 `python -X utf8 work/05_led_calculation.py`을 실행합니다. 저장과 실행은 별도 행동입니다.

```
supply_v = 3.3
led_v = 2.0
resistance_ohm = 330

if resistance_ohm <= 0 or supply_v < led_v:
    raise ValueError("계산 가정을 확인하세요")

current_ma = (supply_v - led_v) / resistance_ohm * 1000
print("assumed_LED_mA:", round(current_ma, 2))
```

예상 출력:

```
assumed_LED_mA: 3.94
```

1~3행은 공급 전압·가정 LED 강하·저항을 이름에 저장합니다. =는 오른쪽 값을 왼쪽 이름에 붙이는 기호입니다. 4행의 <=는 작거나 같음, <는 작음, or는 둘 중 하나라도 참이라는 뜻입니다. 콜론 뒤 들여쓴 5행은 잘못된 가정을 오류로 알립니다. 6행의 괄호는 뺄셈을 먼저 하게 하며 /는 나눗셈, *는 곱셈입니다. 1000은 A를 mA로 바꾸는 수입니다. 7행은 round로 소수 둘째 자리까지 반올림해 출력합니다. 저항을 660으로 바꾸면 1.97입니다. 저항 0은 오류이므로 330으로 복구합니다. 계산 실행에는 보드와 전원을 연결하지 않습니다.

LED는 전류가 흐를 때 빛을 내는 다이오드이며 방향과 동작 조건이 있습니다. 직렬 저항은 회로 전류를 제한하는 데 사용됩니다. LED의 전압 강하는 부품과 전류에 따라 달라집니다.

같은 330Ω이라도 저항 양단 전압이 달라지면 전류가 달라집니다. LED가 있는 회로를 저항 하나의 계산과 구분합니다.

입력에서 살펴볼 예

종이 가정: 공급 3.3V LED 전압 강하 2.0V 저항 330Ω $I = (3.3 - 2.0) / 330$

가정하에서 약 3.94mA입니다. 2.0V는 이번 계산용 가정입니다. 실물 LED 사양은 미확인입니다.

구분해야 할 반례

저항 없이 LED를 직접 연결하라는 지시가 아닙니다. LED모델·정격·극성·핀 허용 조건을 확인하기 전에는 통전하지 않습니다.

스스로 설명할 질문: A03의10mA와 여기3.94mA가 다른 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A04 · 따라 하기와 작성칸

A04 · LED와 저항의 역할

목적 같은330Ω이라도 저항 양단 전압이 달라지면 전류가 달라집니다. LED가 있는 회로를 저항 하나의 계산과 구분합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A04_input.md](#)

작업: [A04_worksheet.md](#)

단계

1. 저항에 걸리는 전압1.3V를 구합니다.
2. 전류를 mA로 계산합니다.
3. 실물 전에 확인할 문서 항목 세 개를 적습니다.

예상 결과 입력 근거를 표시하고 LED 계산에 판단을 남깁니다.

확인 방법 A03의10mA와 여기3.94mA가 다른 이유는 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A05 · 칩·모듈·개발 보드

마이크로컨트롤러는 프로그램을 실행하며 입출력을 다루는 칩입니다. 모듈은 칩과 주변 부품을 묶고, 개발 보드는 연결과 개발을 돕는 부품을 더합니다. ESP32는 여러 계열을 포함합니다.

같은 ESP32라는 이름만으로 핀 위치·USB·내장LED·전원 경로를 확정할 수 없습니다. 지도 이름과 실제 주소가 다른 것처럼 정확한 모델이 필요합니다.

입력에서 살펴볼 예

확인 카드 칩 계열: 미확인 보드 모델/리비전: 미확인 공식 핀맵: 미확인

부품 이름과 보드 이름을 따로 기록합니다. 사진 글자와 공식 문서의 모델을 맞춘 후 계획합니다.

구분해야 할 반례

인터넷 예제의 GPIO2가 모든 보드의 내장LED라고 가정하지 않습니다. 보드 외형이 비슷해도 핀 역할은 다를 수 있습니다.

스스로 설명할 질문: 모델이 없는데 핀 번호부터 고르면 어떤 근거가 빠지나요?

활동 연결: [입력](#) · [기록 양식](#)

A05 · 따라 하기와 작성칸

A05 · 칩·모듈·개발 보드

목적 같은 ESP32라는 이름만으로 핀 위치·USB·내장LED·전원 경로를 확정할 수 없습니다. 지도 이름과 실제 주소가 다른 것처럼 정확한 모델이 필요합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A05_input.md](#)

작업: [A05_worksheet.md](#)

단계

1. 칩·모듈·보드 역할을 구분합니다.
2. 확인 카드의 빈 항목을 질문으로 바꿉니다.
3. 이번 수업의 미확정 항목을 표시합니다.

예상 결과 입력 근거를 표시하고 보드 확인 카드에 판단을 남깁니다.

확인 방법 모델이 없는데 핀 번호부터 고르면 어떤 근거가 빠지나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A06 · GPIO와 논리 수준

GPIO는 프로그램으로 입출력 역할을 정하는 범용 핀입니다. 디지털 입력은 정해진 전압 범위를 LOW/HIGH로 해석합니다. 논리값은 물리 전압의 약속과 연결되지만 같은 단위는 아닙니다.

입력·출력 방향을 혼동하거나 전압 범위를 넘기면 코드가 맞아도 장치 문제가 생깁니다. GPIO와 공급 핀을 먼저 구분합니다.

입력에서 살펴볼 예

신호 계획 버튼 상태 → 디지털 입력 표시 상태 → 디지털 출력 구체 GPIO 번호 → 미정

pinMode는 방향을 정하는 API입니다. digitalRead는 논리 상태를 읽습니다. 번호·허용 전압은 모델 문서로 확인합니다.

구분해야 할 반례

HIGH가 모든 보드에서 5V라는 뜻은 아닙니다. USB에 5V가 들어간다고 GPIO가 5V 입력을 허용한다고 추정하지 않습니다.

스스로 설명할 질문: 전원 입력과 신호 입력을 왜 구분해야 하나요?

활동 연결: [입력](#) · [기록 양식](#)

A06 · 따라 하기와 작성칸

A06 · GPIO와 논리 수준

목적 입력·출력 방향을 혼동하거나 전압 범위를 넘기면 코드가 맞아도 장치 문제가 생깁니다. GPIO와 공급 핀을 먼저 구분합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A06_input.md](#)

작업: [A06_worksheet.md](#)

단계

1. 신호마다 입력·출력을 표시합니다.
2. pinMode/digitalRead 역할을 연결합니다.
3. 전압·핀맵 확인 전 보류 결정을 적습니다.

예상 결과 입력 근거를 표시하고 입출력 계획에 판단을 남깁니다.

확인 방법 전원 입력과 신호 입력을 왜 구분해야 하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

3교시 · 변환·스키마·모의입력

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 원시값에서 기록까지

먼저 할 활동: A07, A08, A09

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A07 · 아날로그와 ADC

3교시 필수 개념 보충 · 전압 신호를 숫자 기록으로 옮기기

실무 상황과 원리

온도 센서 값을 대시보드에 보내려면 전기 신호가 어떤 숫자와 단위를 뜻하는지 약속해야 합니다. 연속적으로 변하는 전압을 아날로그 신호라 부릅니다. ADC는 그 신호를 제한된 개수의 숫자 단계로 바꾸는 변환기입니다. 숫자가 되었다고 바로 섭씨 온도가 되지는 않습니다.

계산 연습용으로 03.3V를 코드 04095에 대응하는 이상적인 선형 모델을 가정합니다. 값 2048의 전압을 끝점 대응식으로 계산하면 $2048 \div 4095 \times 3.3 \approx 1.6504V$ 입니다. 이는 실물 ESP32 ADC의 교정 결과가 아닙니다. 실제 분해능 설정, 감쇠, 비선형성, 센서 변환식과 오차를 확인해야 합니다. 1.65V를 1.65°C라 적는 것은 단위가 생략된 해석 오류입니다.

이번 파일 실습은 이미 섭씨 단위로 만든 temperature_c를 읽습니다. JSON 한 기록은 sensor_id, seq, temperature_c처럼 이름과 값의 짝으로 표현합니다. { }는 한 기록의 묶음, 콜론은 이름과 값의 연결, 쉼표는 항목 구분입니다. null은 값이 없음을 나타내며 Python으로 읽으면 None이 됩니다. 완성 예의 입력 seq=1, temperature_c=27.8은 숫자 확인 → 28.0 이상 비교 → normal 출력으로 이어집니다. 입력값 "27.8"은 따옴표가 있는 문자열이라 현재 예제에서 invalid입니다. missing, invalid, alert, normal의 출력 이름은 수업용 판단 약속이며 실제 센서 고장 원인을 확정하지 않습니다. 원시 입력과 판정 결과를 함께 남겨야 재검토할 수 있습니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 가장 기록 세 개의 temperature_c가 0, null, "0"일 때 숫자 온도, 미측정, 잘못된 형식을 구분하세요. ADC 코드와 섭씨 값 사이에 필요한 정보 한 가지도 쓰세요.

정답과 오답의 이유

0은 숫자, null은 미측정, "0"은 문자열이라 현재 계약에서는 invalid입니다. 센서 변환식과 교정 조건이 필요합니다. 결측을 0으로 채우면 실제 영도 측정과 구별되지 않는 새 오류가 생깁니다.

확인 질문과 답

1. ADC 값은 바로 온도인가요? → 아닙니다.
2. JSON null의 Python 값은? → None입니다.
3. 2048 계산은 실물 교정인가요? → 이상적 모델의 연습입니다.

아날로그 신호는 범위 안에서 연속적으로 변할 수 있는 물리 신호입니다. ADC는 입력 전압을 정해진 해상도의 디지털 수로 변환합니다. 눈금자로 길이를 읽듯 유한 단계로 기록합니다.

변환 수치에는 해상도·입력 범위·보정 조건이 있습니다. 큰 숫자가 곧 온도라는 뜻은 아닙니다.

입력에서 살펴볼 예

교육용 12비트 가정: 0~4095 analogRead: 원시 변환값 analogReadMilliVolts: 보정된 mV 값

원시 수치와mV, 온도°C는 다른 단계입니다. 실제 범위와 해상도는 칩·설정에 따라 확인합니다.

구분해야 할 반례

raw/4095×3.3을 모든 ESP32의 정확한 전압 보정식으로 쓰지 않습니다. ADC 감쇠·특성·보정과 센서 변환식이 필요합니다.

스스로 설명할 질문: ADC 값2048이 곧20.48°C라는 결론에 무엇이 빠졌나요?

활동 연결: [입력](#) · [기록 양식](#)

A07 · 따라 하기와 작성칸

A07 · 아날로그와 ADC

목적 변환 수치에는 해상도·입력 범위·보정 조건이 있습니다. 큰 숫자가 곧 온도라는 뜻은 아닙니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A07_input.md](#)

작업: [A07_worksheet.md](#)

단계

1. 원시수·mV·°C를 단계로 정리합니다.
2. 각 단계에 필요한 설정을 씁니다.
3. 정확도가 필요한 검증 항목을 적습니다.

예상 결과 입력 근거를 표시하고 변환 단계에 판단을 남깁니다.

확인 방법 ADC 값2048이 곧20.48°C라는 결론에 무엇이 빠졌나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A08 · 센서 기록의 스키마

센서는 물리량을 전기 신호 등으로 바꾸는 장치입니다. 기록 스키마는 식별자·순번·값·단위·상태의 약속입니다. 포장 상자의 라벨처럼 값의 뜻을 같이 보냅니다.

센서에서 온 숫자만 저장하면 어느 장치·어느 순서·어떤 단위인지 잃습니다. 값 없음도 정상0과 구별해야 합니다.

입력에서 살펴볼 예

seq1:27.5°C seq2:28.0°C seq3:null seq4:0.0°C seq5:"29.0" 문자열

null은 값 없음입니다. 0.0은 실제 수치입니다. "29.0"은 이 스키마에서 잘못된 타입입니다.

구분해야 할 반례

센서가 응답하지 않았다고0으로 채우면 실제0과 구분할 수 없습니다. 누락을 정상 측정처럼 만들지 않습니다.

스스로 설명할 질문: 0과null을 바꾸면 어떤 판단이 달라지나요?

활동 연결: [입력](#) · [기록 양식](#)

A08 · 따라 하기와 작성칸

A08 · 센서 기록의 스키마

목적 센서에서 온 숫자만 저장하면 어느 장치·어느 순서·어떤 단위인지 잃습니다. 값 없음도 정상0과 구별해야 합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A08_input.md](#) · [sensor_samples.json](#)

작업: [A08_worksheet.md](#)

단계

1. 실제JSON의 각 자료형을 표시합니다.
2. 단위와 식별자 열을 확인합니다.
3. 값 없음과 형식 오류를 다른 칸에 적습니다.

예상 결과 입력 근거를 표시하고 스키마와 타입에 판단을 남깁니다.

확인 방법 0과null을 바꾸면 어떤 판단이 달라지나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A09 · 모의 센서 파일을 읽기

모의 입력은 실제 장치 대신 조건을 통제된 값입니다. 프로그램의 분기와 출력은 시험할 수 있지만 센서의 오차나 연결 품질은 시험하지 못합니다.

같은 입력으로 조건 변경 전후를 비교할 수 있어 실물 연결 전 처리 논리를 확인하기 좋습니다.

입력에서 살펴볼 예

02_sensor_replay.py 기준 28.0°C 이상 alert 숫자 이외는 invalid null은 missing

1 normal · 2 alert 3 missing · 4 normal 5 invalid

구분해야 할 반례

출력 alert는 팬이나 릴레이를 작동시킨 결과가 아닙니다. 화면용 상태 이름이며 실제 제어 명령이 없습니다.

스스로 설명할 질문: 모의 입력으로 확인한 것과 못 한 것은 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

02_sensor_replay.py 전체

```
import json

from pathlib import Path

root = Path(__file__).resolve().parent.parent
source = root / "data" / "sensor_samples.json"

samples = json.loads(source.read_text(encoding="utf-8"))

for sample in samples:
    value = sample["temperature_c"]
    if value is None:
        state = "missing"
    elif type(value) not in (int, float):
        state = "invalid"
    elif value >= 28.0:
        state = "alert"
    else:
        state = "normal"
    print(sample["seq"], value, state)
```

실행 조건: Python 3 표준 라이브러리만 사용합니다. 장치·포트·전원을 연결하지 않습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/02_sensor_replay.py`

결과:

```
1 27.5 normal
2 28.0 alert
3 None missing
4 0.0 normal
5 29.0 invalid
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 코드의 28.0을 29.0으로 바꾸면 2번은 normal이 됩니다. 문자열 "29.0"은 계속 invalid입니다.

오류 복구: JSON이 깨지면 큰따옴표와 쉼표를 복구합니다. 값 없음과 0을 바꿔 오류를 숨기지 않습니다.

A09 · 따라 하기와 작성칸

A09 · 모의 센서 파일을 읽기

목적 같은 입력으로 조건 변경 전후를 비교할 수 있어 실물 연결 전 처리 논리를 확인하기 좋습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A09_input.md](#) · [02_sensor_replay.py](#) · [sensor_samples.json](#)

작업: [A09_worksheet.md](#)

단계

1. 코드 분기 순서를 읽고 출력5개를 예상합니다.
2. 실행 또는 제공 출력과 대조합니다.
3. 경계29.0 수정 결과와 미확인 범위를 씁니다.

예상 결과 입력 근거를 표시하고 예상과 출력에 판단을 남깁니다.

확인 방법 모의 입력으로 확인한 것과 못 한 것은 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

4교시 · 경계·흔들림·기록 보존

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 경계·흔들림·원시값

먼저 할 활동: A10, A11, A12

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A10 · 조건의 경계와 결측

4교시 필수 개념 보충 · 경계값과 상태를 기억하는 판단

실무 상황과 원리

가상 온도가 27.9와 28.1 사이를 오가면 경보가 켜졌다 꺼졌다 반복될 수 있습니다. 한 문턱만 사용하는 규칙은 현재 값만 봅니다. 서로 다른 켜짐과 꺼짐 문턱을 사용하는 히스테리시스는 이전 상태도 기억합니다. 실제 장비 제어에 적용하려면 별도의 위험과 정격 검토가 필요하며 오늘은 상태 계산만 합니다.

제작 규칙은 28.0 이상이면 alert=True, 27.5 이하이면 False, 사이이면 이전 상태 유지입니다. 초기 상태 False와 입력 [27.8,28.1,27.9,27.4]를 차례로 추적하면 False,True,True,False입니다. 27.9에서 False로 바뀌지 않는 이유는 중간 구간이기 때문입니다. 28.0과 27.5는 각각 켜짐과 꺼짐에 포함됩니다. 초기 상태가 달라지면 첫 중간 값의 출력도 달라지므로 시작 조건을 기록합니다.

None은 온도가 아니라 측정 누락입니다. 예제는 missing과 no command를 출력하고 이전 내부 상태를 유지합니다. 이는 통신 실패 시 실제 부하를 안전하게 제어하는 정책이 완성되었다는 뜻은 아닙니다. 보존한 상태와 현재 유효한 측정이 있다는 사실을 구분합니다. 완성 추적표에는 순번, 원시값, 이전 상태, 적용한 조건, 새 상태, 측정 유효 여부를 둡니다. elif는 앞 조건이 거짓인 경우에만 다음 조건을 확인하고, continue는 이번 반복의 나머지 처리를 건너뛸니다. 결측 행을 지우거나 이전 온도를 새 측정처럼 복사하면 왜 판단이 유지되었는지 알 수 없습니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 초기 False, 새 입력 [28.0,27.7,None,27.5]의 상태와 결측 시 출력 메시지를 순서대로 작성하세요. 중간 27.7이 True인 이유도 설명하세요.

정답과 오답의 이유

True,True,상태 True 유지와 no command,False입니다. 27.7은 두 문턱 사이입니다. None을 영도로 바꾸어 꺼짐 조건에 넣는 답은 결측과 수치를 혼동한 오답입니다. 실제 출력 장치의 안전 정책은 아직 시험하지 않았습니다.

확인 질문과 답

1. 중간 구간은? → 이전 상태를 유지합니다.
2. 초기 상태가 필요한 이유는? → 첫 중간 값의 판단이 달라질 수 있습니다.
3. 결측 출력은 새 온도인가요? → 아닙니다.

조건 경계는 처리 결과가 바뀌는 지점입니다. >=는 기준과 같아도 참입니다. 결측과 잘못된 자료형을 먼저 걸러야 숫자 비교를 정확히 설명할 수 있습니다.

평소 값만 시험하면 같음·이상·없음의 차이를 놓칩니다. 문턱 바로 앞·위·없는 입력을 나란히 둡니다.

입력에서 살펴볼 예

기준28.0 27.9 / 28.0 / 28.1 null / "28.0" / true

27.9 normal 28.0·28.1 alert null missing · 문자/불리언 invalid

구분해야 할 반례

Python에서bool은int의 하위 타입이지만 이 센서 스키마에서는 온도가 아닙니다. 제공 코드는 type을 엄격히 검사합니다.

스스로 설명할 질문: 28.0과문자열"28.0"이 다른 이유를 말할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

A10 · 따라 하기와 작성칸

A10 · 조건의 경계와 결측

목적 평소 값만 시험하면 같음·이상·없음의 차이를 놓칩니다. 문턱 바로 앞·위·없는 입력을 나란히 둡니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A10_input.md](#)

작업: [A10_worksheet.md](#)

단계

1. 여섯 입력의 예상 상태를 씁니다.
2. 숫자·문자·불리언을 분류합니다.
3. 검사 순서와 근거를 설명합니다.

예상 결과 입력 근거를 표시하고 경계 표에 판단을 남깁니다.

확인 방법 28.0과문자열"28.0"이 다른 이유를 말할 수 있나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A11 · 흔들림과 히스테리시스

히스테리시스는 이전 상태에 따라 현재 판단이 달라지는 특성입니다. 여기서는 켜기28.0,끄기27.5로 기준을 나눠 작은 흔들림에서 반복 전환을 줄입니다.

경계 하나만 쓰면27.9와28.1이 번갈아올 때 상태가 계속 바뀔 수 있습니다. 표시 안정화의 원리를 모의 값으로 살펴봅니다.

입력에서 살펴볼 예

27.8 → 28.1 → 27.9 → 28.2 → 27.4 → null → 28.3 켜기 $\geq$ 28.0 / 끄기 $\leq$ 27.5

꺼짐 → 켜짐 → 켜짐 → 켜짐 → 켜짐 → 명령 없음 → 켜짐

구분해야 할 반례

이 예의 누락 처리가 모든 장치의 안전 정책은 아닙니다. 실제 제어는 장치 특성에 맞는 별도 실패 안전 정책이 필요합니다.

스스로 설명할 질문: 27.9에서 직전 상태를 왜 기억하나요?

활동 연결: [입력](#) · [기록 양식](#)

03_hysteresis.py 전체

```
values = [27.8, 28.1, 27.9, 28.2, 27.4, None, 28.3]

alert = False

for value in values:
    if value is None:
        print("missing", "no command")
        continue
    if value >= 28.0:
        alert = True
    elif value <= 27.5:
        alert = False
    print(value, alert)
```

실행 조건: Python 3 표준 라이브러리만 사용합니다. 장치·포트·전원을 연결하지 않습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/03_hysteresis.py`

결과:

```
27.8 False
28.1 True
27.9 True
28.2 True
27.4 False
missing no command
28.3 True
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 꺼짐 기준 27.5를 27.9로 바꾸면 세 번째 27.9에서 False가 됩니다.

오류 복구: 상태 변수를 반복문 안에서 매번 False로 초기화하면 이력 효과가 사라집니다. 들여쓰기와 초기화 위치를 확인합니다.

A11 · 따라 하기와 작성칸

A11 · 흔들림과 히스테리시스

목적 경계 하나만 쓰면27.9와28.1이 번갈아올 때 상태가 계속 바뀔 수 있습니다. 표시 안정화의 원리를 모의 값으로 살펴봅니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A11_input.md](#) · [03_hysteresis.py](#)

작업: [A11_worksheet.md](#)

단계

- 1. 상태를 순서대로 예상합니다.
- 2. 완성 코드를 실행하거나 출력과 비교합니다.
- 3. 끄기 기준27.9의 변화를 기록합니다.

예상 결과 입력 근거를 표시하고 상태 추적에 판단을 남깁니다.

확인 방법 27.9에서 직전 상태를 왜 기억하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A12 · 원시값과 판단을 함께 보존

처리 결과는 원문을 바꾼 값이 아니라 원문으로부터 얻은 판단입니다. 수업에서는 raw값과 상태·규칙을 함께 기록합니다. 영수증 원문과 정산 결과를 따로 남기는 것과 같습니다.

나중에 기준이 달라져도 원래 값을 다시 판단할 수 있어야 합니다. 잘못된 입력을 지워 버리면 문제 원인을 찾기 어렵습니다.

입력에서 살펴볼 예

원문: seq5,"29.0" 판단: invalid 규칙: 온도는 수 또는null 처리: 원문 보존·확인 대기

수정 근거가 없으므로 숫자29.0으로 고치지 않습니다. 원시값과 판단을 별도 칸에 남깁니다.

구분해야 할 반례

alert를 없애려고 온도를 낮추거나 invalid를0으로 바꾸는 것은 검수가 아닙니다. 확인된 정정만 이력과 함께 적용합니다.

스스로 설명할 질문: 결과 상태만 저장하면 무엇을 잃나요?

활동 연결: [입력](#) · [기록 양식](#)

A12 · 따라 하기와 작성칸

A12 · 원시값과 판단을 함께 보존

목적 나중에 기준이 달라져도 원래 값을 다시 판단할 수 있어야 합니다. 잘못된 입력을 지워 버리면 문제 원인을 찾기 어렵습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A12_input.md](#)

작업: [A12_worksheet.md](#)

단계

1. raw·state·rule 칸을 채웁니다.
2. 정정 근거 없는 항목은 보류로 표시합니다.
3. 확인 요청 질문을 구체적으로 씁니다.

예상 결과 입력 근거를 표시하고 검수 기록에 판단을 남깁니다.

확인 방법 결과 상태만 저장하면 무엇을 잃나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

5교시 · 펌웨어·통신·배선 검토

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 펌웨어·통신·배선 검토

먼저 할 활동: A13, A14, A15

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A13 · setup·loop와 직렬 출력

5교시 필수 개념 보충 · 펌웨어 반복과 직렬 통신의 경계

실무 상황과 원리

PC 파일에서 한 줄이 출력되는 것을 보고 센서가 연결되었다고 생각하기 쉽습니다. 그러나 Python 파일 재생과 보드 펌웨어는 실행하는 컴퓨터가 다릅니다. 펌웨어는 보드에 넣어 실행하는 프로그램이며 Arduino 방식의 setup은 시작할 때 한 번, loop는 그 뒤 반복되는 함수입니다.

제공 04_serial_mock.ino에서 setup은 Serial.begin(115200)으로 통신 속도를 설정합니다. loop는 고정 JSON 문장을 출력하고 delay(1000)으로 1000밀리초를 기다립니다. 1000밀리초는 1초지만 다른 처리 시간도 있으므로 정확한 1초 주기를 측정할 결과라고 부르지 않습니다. 고정 28.0을 출력하는 코드는 센서 측정 함수가 없습니다. 출력 문자열이 온도처럼 보여도 실제 물리량을 읽은 근거는 없습니다.

직렬 통신은 두 장치가 약속한 설정으로 바이트를 주고받는 경로입니다. 보드와 PC의 속도 설정이 맞지 않으면 글자가 깨질 수 있습니다. JSON 문자열 안의 따옴표는 코드 문자열을 끝내지 않도록 역슬래시로 구분합니다. { }는 함수의 문장 묶음, 세미콜론은 C++ 문장의 끝입니다. Python 코드와 기호 약속이 다르므로 파일 확장자 .ino와 .py도 구분합니다. 완성 확인 순서는 정확한 보드 모델 확인 → 개발 환경과 포트 확인 → 컴파일 → 업로드 → 모니터의 같은 속도 설정 → 원문 관찰입니다. 이 과정은 실물 조건이 갖춰진 경우의 후속 경로이며 이번 파일 재생 검증과 나눠 기록합니다. 배선 검토는 전원을 끈 상태의 도면과 사양 확인부터 시작합니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 고정 문장 출력을 센서 측정으로 바꾸려면 어떤 입력 단계가 빠져 있는지 쓰세요. 모니터에 글자가 깨질 때와 포트 자체가 없을 때 확인 순서를 구분하세요.

정답과 오답의 이유

실제 센서를 읽고 단위로 변환하는 단계가 빠져 있습니다. 깨진 문자는 양쪽 속도 설정부터, 포트 없음은 데이터 케이블·드라이버·장치 인식부터 확인합니다. Python 출력 성공을 보드 업로드 성공이라고 쓰면 실행 위치를 혼동한 것입니다.

확인 질문과 답

1. setup 실행 횟수는? → 시작 때 한 번입니다.
2. 고정 28.0은 측정인가요? → 아닙니다.

3. 115200의 역할은? → 직렬 통신 속도 설정입니다.

펌웨어는 장치에서 실행하는 프로그램입니다. Arduino의 setup은 시작 시 한 번,loop는 이어서 반복 실행됩니다. 직렬 출력은 값을 순서대로 보내는 통신 경로입니다.

PC의 Python 파일과 장치 펌웨어는 실행 위치·도구·종료 방법이 다릅니다. 전체 프로그램의 시작과 반복을 먼저 읽습니다.

입력에서 살펴볼 예

04_serial_mock.ino setup: Serial.begin(115200) loop: 고정 JSON 출력, delay(1000)

설정은 한 번, 출력과 기다림은 반복합니다. 고정 문장 전송이며 센서 측정이 아닙니다.

구분해야 할 반례

delay1000만으로 정확히 1Hz 측정을 보장하지 않습니다. 실행 시간과 통신 등 지연이 더해지고 실제 센서를 읽는 코드도 없습니다.

스스로 설명할 질문: 이 코드가 센서를 읽었다고 말할 수 없는 근거는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

04_serial_mock.ino 전체

```
void setup() {
    Serial.begin(115200);
}

void loop() {
    Serial.println("{\"sensor_id\":\"S01\",\"temperature_c\":28.0}");
    delay(1000);
}
```

실행 조건: ESP32 계열의 정확한 보드와 core, USB/UART Serial 경로가 확인된 후 강사가 선택합니다. GPIO 핀을 쓰지 않습니다.

실행 폴더: examples/course

명령: 보드 확정 뒤 Arduino IDE에서 보드·포트·core 설정 후 빌드/업로드하는 선택 경로. 이번 제작에서는 미실행.

결과:

```
조건부 예상: {"sensor_id":"S01","temperature_c":28.0}
같은 문장을 반복 출력. 실물 컴파일·업로드·수신 미실행.
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 온도 리터럴28.0을29.0으로 바꾸면 전송 문장의 값만 바뀝니다. 실제 센서를 측정한 것이 아닙니다.

오류 복구: 보드 모델·core·USB 모드·Serial 연결 경로를 확인합니다. 포트가 없으면 연결을 추측하지 말고 Python 모의 입력으로 진행합니다.

A13 · 따라 하기와 작성칸

A13 · setup·loop와 직렬 출력

목적 PC의 Python 파일과 장치 펌웨어는 실행 위치·도구·종료 방법이 다릅니다. 전체 프로그램의 시작과 반복을 먼저 읽습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A13_input.md](#) · [04_serial_mock.ino](#)

작업: [A13_worksheet.md](#)

단계

1. 함수와 종괄호 블록을 표시합니다.
2. 문자열 내부 따옴표와 세미콜론 뜻을 읽습니다.
3. 예상 출력과 보드 실행 미확인을 구분합니다.

예상 결과 입력 근거를 표시하고 실행 흐름에 판단을 남깁니다.

확인 방법 이 코드가 센서를 읽었다고 말할 수 없는 근거는 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A14 · 시리얼과 파일 재생 경계

시리얼 수신은 실제 연결에서 바이트를 받아 기록하는 일이고 파일 재생은 이미 저장한 내용을 읽는 일입니다. 택배를 실제 받는 과정과 받은 목록을 읽는 과정을 구분하는 비유입니다.

파일 성공으로 포트·전송속도·줄바꿈·단절 처리를 확인했다고 말하지 않아야 합니다. 각 단계의 관찰 근거를 적습니다.

입력에서 살펴볼 예

이번: 파일 JSON 읽기 성공 선택: 보드 직렬 수신 추가 확인: 포트·속도·줄바꿈·끊김

115200은 설정값입니다. 보드의 Serial 경로는 USB 모드와 모델 조건에 따릅니다.

구분해야 할 반례

COM3 같은 포트명을 모든 PC에 적용하지 않습니다. 수신기가 없는데 출력이 보이지 않는 일을 센서 고장으로 단정하지 않습니다.

스스로 설명할 질문: 파일과 실제 수신의 시험 항목을 어떻게 나누나요?

활동 연결: [입력](#) · [기록 양식](#)

A14 · 따라하기와 작성칸

A14 · 시리얼과 파일 재생 경계

목적 파일 성공으로 포트·전송속도·줄바꿈·단절 처리를 확인했다고 말하지 않아야 합니다. 각 단계의 관찰 근거를 적습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A14_input.md](#)

작업: [A14_worksheet.md](#)

단계

- 1. 파일 경로에서 확인한 것을 씁니다.
- 2. 통신 확인 카드의 미확인 칸을 채웁니다.
- 3. 끊김 복구를 실물 없이 순서로 설명합니다.

예상 결과 입력 근거를 표시하고 검증 경계에 판단을 남깁니다.

확인 방법 파일과 실제 수신의 시험 항목을 어떻게 나누나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A15 · 배선 전 검토표 작성

배선 검토표는 연결 전에 신호·전원·기준·정격·핀 근거를 확인하는 문서입니다. 공사를 시작하기 전 도면을 검토하듯 확인하지 못한 항목은 미정으로 남깁니다.

추측한 핀이나 전압을 실행 계획으로 바꾸지 않습니다. 이번 수업은 종이 연결 계획과 확인 질문까지 작성하며 통전하지 않습니다.

입력에서 살펴볼 예

항목: 보드 모델,핀맵,전원,신호 전압,센서 모델,극성 현재: 모두 장비 정보 미확정

실제 핀 번호는 빈칸으로 남깁니다. 공식 문서의 확인 위치를 적습니다. 통전 결정은 보류합니다.

구분해야 할 반례

목표가 있다고 연결을 추측해서 완성하지 않습니다. 젖은 손·손상 케이블·금속 단락·전원 연결 중 배선 변경은 피하는 기본 원칙을 적용합니다.

스스로 설명할 질문: 미정 표시는 실패가 아니라 어떤 역할을 하나요?

활동 연결: [입력](#) · [기록 양식](#)

A15 · 따라 하기와 작성칸

A15 · 배선 전 검토표 작성

목적 추측한 핀이나 전압을 실행 계획으로 바꾸지 않습니다. 이번 수업은 종이 연결 계획과 확인 질문까지 작성하며 통전하지 않습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A15_input.md](#) · [wiring-review.md](#)

작업: [A15_worksheet.md](#)

단계

1. 검토표 각 항목의 필요 이유를 씁니다.
2. 근거 없는 핀 번호와 전압 추정을 지웁니다.
3. 강사가 확인할 순서를 정합니다.

예상 결과 입력 근거를 표시하고 배선 검토표에 판단을 남깁니다.

확인 방법 미정 표시는 실패가 아니라 어떤 역할을 하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

6교시 · 새 요구·종합 검수·인수인계

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 새 센서 요구와 모의 검수

먼저 할 활동: A16, A17, A18

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A16 · 새 센서 요구를 시험으로 바꾸기

6교시 필수 개념 보충 · 모의 검증과 실물 인수인계의 증거

실무 상황과 원리

다음 팀이 온도 상태 표시를 이어 만들려 합니다. “코드 완료”라는 한 줄로 넘기면 어떤 입력이 시험되었고 어떤 장비가 빠져 있는지 알 수 없습니다. 인수인계는 이어서 판단할 사람이 같은 근거를 재현할 수 있도록 입력, 처리 규칙, 출력과 미확인을 묶는 일입니다.

완성 예의 요구는 숫자 온도가 28.0 이상일 때 alert를 출력하고 null일 때 missing을 출력하는 것입니다. 시험 입력 [27.9,28.0,None,"28.0"]의 기대는 normal,alert,missing,invalid입니다. 로컬 파일 실행에서 이 네 결과가 관찰되었다면 해당 비교는 통과입니다. 실제 센서의 측정 정확도, 신호 전압, 직렬 전송, 보드 재시작과 출력 부하의 반응은 별도 시험이며 장비가 없으면 미실행입니다. 모의 값은 제어 로직의 경계를 시험하기 좋지만 물리 장치의 정확성을 대신하지 않습니다.

요구를 “새 온도 경계 30.0”으로 바꾸면 코드 한 숫자만 바꾸는 데서 끝내지 않습니다. 29.9,30.0,30.1과 결측을 포함한 새 기대값을 먼저 쓰고 시험표와 표시 설명을 갱신합니다. 히스테리시스가 있다면 꺼짐 문턱과 초기 상태도 다시 합의합니다. 전달 묶음에는 원본 모의 데이터, 코드, 실행 명령, 관찰 출력, 경계 시험표, 모델·전압·핀 확인표를 넣습니다. 실물 조건 표의 빈칸은 추측으로 채우지 않습니다. 실제 장비를 사용한 사람과 측정 방법이 있어야 실물 검증 상태를 바꿀 수 있습니다. 파일 성공과 장치 성공을 같은 칸에 적지 않습니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 다음 팀이 “센서 오차 $\pm 0.5^{\circ}\text{C}$ 도 검증된 것인가?”라고 물었습니다. 현재 근거로 답변을 작성하고 추가 시험의 입력·비교 기준·필요 장비를 제안하세요.

정답과 오답의 이유

현재는 모의 판단만 검증했고 실물 오차는 미확인입니다. 기준 온도계와 실제 센서를 같은 조건에서 비교하고 여러 온도 구간의 차이를 기록하는 시험을 제안합니다. 모의 28.0의 출력 일치 센서 정확도 증거가 아닙니다.

확인 질문과 답

1. 모의 실행이 확인한 것은? → 주어진 값에 대한 처리 규칙입니다.
2. 새 경계 요청이면? → 기대값과 시험도 갱신합니다.
3. 실물 조건이 빈칸이면? → 미확인으로 남깁니다.

요구는 누가 어떤 상황에서 어떤 결과가 필요한지 말하는 문장입니다. 시험은 그 결과를 구체 입력과 관찰로 확인하는 절차입니다.

좋은 구현보다 먼저 모호한 말을 검사할 수 있는 조건으로 바꿉니다. 실제 센서가 없어도 표시 규칙 시험은 설계할 수 있습니다.

입력에서 살펴볼 예

요구: 창고 온도가 높으면 알려줘 합의 제안: 숫자28.0이상alert null→missing,문자→invalid 출력만 표시,장치 제어 없음
 경계27.9~28.0과null·문자를 시험합니다. 실제 위험 온도 기준은 현장 담당 확인이 필요합니다.

구분해야 할 반례

28.0은교육용 규칙입니다. 모든 창고나 물품의 안전 기준이라고 설명하지 않습니다.

스스로 설명할 질문: 요구·제작 가정·검증 결과를 어떻게 구분하나요?

활동 연결: [입력](#) · [기록 양식](#)

A16 · 따라 하기와 작성칸

A16 · 새 센서 요구를 시험으로 바꾸기

목적 좋은 구현보다 먼저 모호한 말을 검사할 수 있는 조건으로 바꿉니다. 실제 센서가 없어도 표시 규칙 시험은 설계할 수 있습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A16_input.md](#)

작업: [A16_worksheet.md](#)

단계

1. 모호한 말을 조건 문장으로 바꿉니다.
2. 입력4개의예상 상태를 작성합니다.
3. 현장확인 필요 조건을 별도로 씁니다.

예상 결과 입력 근거를 표시하고 요구와 시험에 판단을 남깁니다.

확인 방법 요구·제작 가정·검증 결과를 어떻게 구분하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A17 · 종합 모의 검수

종합 검수는 새로운 입력에 같은 규칙을 적용하고출력·원시값·미확인 항목을 함께 남기는 일입니다. 프로그램의 성공 메시지와 데이터의 의미를 따로 확인합니다.

익숙한 다섯 입력만 외우지 않고 새 순서에서0·경계·미측정·형식 오류를 구분합니다.

입력에서 살펴볼 예

새 입력: 0.0 /27.9 /28.0 /null /"28.1" 기준28.0 source를sensor_new.json으로수정

예상: normal,normal,alert,missing,invalid 센서 실측이 아니라 파일 모의입니다.

구분해야 할 반례

실행이 정상 종료되어도 invalid항목은 남아 있을 수 있습니다. 원문 타입 오류를 검수 결과에 기록합니다.

스스로 설명할 질문: 정상 종료와 모든 측정값 정상의 차이를 설명할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

새 입력의 예상과 실제를 나란히 기록

모의 기준은 28.0입니다. N=정상, A=경계 이상 알림, M=미측정, I=잘못된 자료형입니다. 상태는 실제 보드 측정 결과가 아닙니다.

입력 값(따옴표 보존)	자료형	예상 상태·이유	실제 상태	확인 방법(직접 실행/제공 출력 관찰)
0.0	_____	_____	_____	_____
27.9	_____	_____	_____	_____
28.0	_____	_____	_____	_____
null	_____	_____	_____	_____
"28.1"	_____	_____	_____	_____

- 사용 입력·규칙·출력 위치: _____
- 프로그램 정상 종료 여부: _____
- 정상 종료와 모든 측정값이 정상이라는 말이 다른 이유: _____
- 실제 장비에서 미확인인 모델·전압·핀·연결 자료: _____
- 확인 전 보류할 항목과 확인할 자료: _____

A17 · 따라 하기와 작성칸

A17 · 종합 모의 검수

목적 익숙한 다섯 입력만 외우지 않고 새 순서에서 0·경계·미측정·형식 오류를 구분합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A17_input.md](#) · [sensor_new.json](#) · [02_sensor_replay.py](#)

작업: [A17_worksheet.md](#)

단계

1. 새 입력의 상태를 먼저 예상합니다.
2. work복사 코드의 입력 파일명만 바꿔 비교합니다.
3. 원시값·출력·미확인장비 조건을 제출합니다.

예상 결과 입력 근거를 표시하고 종합 예상·결과에 판단을 남깁니다.

확인 방법 정상 종료와 모든 측정값 정상의 차이를 설명할 수 있나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

새 입력의 예상과 실제를 나란히 기록

모의 기준은 28.0입니다. N=정상, A=경계 이상 알림, M=미측정, I=잘못된 자료형입니다. 상태는 실제 보드 측정 결과가 아닙니다.

입력 값(다음표 보존)	자료형	예상 상태·이 유	실제 상태	확인 방법(직접 실행/제공 출력 관찰)
0.0	_____	_____	_____	_____
27.9	_____	_____	_____	_____

입력 값(따옴표 보존)	자료형	예상 상태·이유	실제 상태	확인 방법(직접 실행/제공 출력 관찰)
28.0	_____	_____	_____	_____
null	_____	_____	_____	_____
"28.1"	_____	_____	_____	_____

- 사용 입력·규칙·출력 위치: _____
- 프로그램 정상 종료 여부: _____
- 정상 종료와 모든 측정값이 정상이라는 말이 다른 이유: _____
- 실제 장비에서 미확인인 모델·전압·핀·연결 자료: _____
- 확인 전 보류할 항목과 확인할 자료: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A18 · 근거를 남겨 다음 팀에 전달

인수인계는 확인한 사실과 아직 확인하지 못한 조건을 다른 사람이 이어갈 수 있게 적는 일입니다. 결과 숫자만이 아니라 입력·코드·설정·판단 근거가 필요합니다.

다음 팀이 장비를 연결하기 전에 어떤 시험을 추가해야 하는지 알 수 있게합니다. 수업의 완성은 정확한 경계 설명도 포함합니다.

입력에서 살펴볼 예

확인: 파일 판단과 수정·오류경로 미확인: 실물회로·보드빌드·수신 다음: 모델 확정→문서검토→별도 시험

수치 단위, 0/null, 모의/실물을 구분합니다. 판단 근거와 보류 항목을 남깁니다.

구분해야 할 반례

실물 미검증을 숨기고 완료라고 적지 않습니다. 반대로 모의 검증 근거도 버리지 말고 재사용할 수 있게 남깁니다.

스스로 설명할 질문: 검증했다는 문장을 어떤 근거와 함께 쓸 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

A18 · 따라 하기와 작성칸

A18 · 근거를 남겨 다음 팀에 전달

목적 다음 팀이 장비를 연결하기 전에 어떤 시험을 추가해야 하는지 알 수 있게합니다. 수업의 완성은 정확한 경계 설명도 포함합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A18_input.md](#)

작업: [A18_worksheet.md](#)

단계

1. 전압·전류·저항을 한 문장씩 설명합니다.
2. 종합 결과에 입력 파일과 규칙을 연결합니다.
3. 장비 확정 후 필요한 시험 세 개를 적습니다.

예상 결과 입력 근거를 표시하고 핵심 설명에 판단을 남깁니다.

확인 방법 검증했다는 문장을 어떤 근거와 함께 쓸 수 있나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

용어와 코드 기호 찾아보기

용어를 처음 만날 때

전압

두 점 사이 전하당 전기 에너지 차이. 단위 V.

전류

시간당 흐르는 전하량. 단위 A, 1A=1000mA.

저항

전류 흐름과 전압 관계를 나타내는 양. 단위 Ω .

GND

회로에서 전압 기준으로 삼는 지점. 보호접지와 항상 같지는 않음.

단락

의도한 부하를 우회하는 낮은 저항 경로가 생기는 일.

LED

전류와 극성 등 조건에 따라 빛을 내는 다이오드.

정격

부품이 사용되도록 지정된 조건이나 한계. 정확한 부품 문서 확인 필요.

마이크로컨트롤러

프로그램과 입출력을 실행하는 작은 컴퓨터 칩.

GPIO

프로그램으로 입출력을 정하는 범용 핀. 모델별 제약 존재.

ADC

아날로그 전압을 정해진 단계의 수로 바꾸는 변환기.

해상도

구별해 기록하는 단계의 수나 세밀함. 정확도와 다름.

펌웨어

장치에서 실행하는 프로그램.

Serial

직렬 통신 경로. 보드와 USB 설정에 따라 실제 경로 확인 필요.

히스테리시스

이전 상태가 현재 판단에 영향을 미치는 성질.

ms

밀리초. 1000ms=1초.

null/None

JSON/Python의 값 없음 표현. 0과 구별.

리터럴

코드에 직접 적어 둔 값. 센서 측정 결과와 구별.

정격·허용 범위

안전하게 적용할 수 있는 부품별 조건. 교육 예시 수치로 대체하지 않음.

비트

0 또는 1을 기록하는 한 자리입니다. 12비트는 가능한 조합이 2의 12제곱, 즉 4096개입니다. 0부터 세면 0~4095로 표현합니다. 단계가 많다는 사실만으로 측정이 정확하다고 말할 수는 없습니다.

컴파일과 업로드

컴파일은 작성한 프로그램을 선택한 보드가 실행할 수 있는 형태로 바꾸는 과정입니다. 업로드는 그 결과를 보드에 보내는 과정입니다. 파일을 저장하는 일과 서로 다릅니다. 이번 자료에서는 보드 모델이 미확정이므로 둘 다 수행하지 않았습니다.

직렬 통신과 바이트

직렬 통신은 데이터를 순서대로 보내는 방식입니다. 바이트는 8비트를 묶은 데이터 단위입니다. Serial로 보낼 글자를 정했다고 실제 연결·수신이 검증된 것은 아닙니다.

core와 포트

Arduino의 보드용 core는 해당 보드에서 프로그램을 만들고 실행하도록 돕는 소프트웨어 묶음입니다. 포트는 PC가 연결 경로를 구별할 때 쓰는 이름입니다. 같은 COM 번호나 USB 설정을 모든 PC와 보드에 적용하지 않습니다.

극성·직렬·부하

극성은 전원이나 부품에서 구분하는 양·음 또는 방향입니다. 직렬 연결은 부품을 하나의 경로에 이어 같은 전류가 지나도록 연결하는 방식입니다. 부하는 회로에서 전기 에너지를 사용하는 대상입니다. 오늘의 LED 경로는 연결 관계를 읽는 종이 예이며 실제 배선 지시가 아닙니다.

감쇠·보정·정확도

ADC의 감쇠 설정은 입력 신호를 변환기가 다룰 범위에 맞게 조절하는 조건입니다. 보정은 변환값과 기준의 차이를 줄이기 위한 대응 관계를 적용하는 일입니다. 정확도는 결과가 기준값에 얼마나 가까운지를 말하며, 기록 단계의 수인 해상도와 다릅니다.

Hz와 리비전

Hz는 1초당 반복 횟수입니다. delay(1000)은 약 1000ms를 기다리라는 명령이며 다른 처리 시간까지 없애 주지 않습니다. 리비전은 같은 제품의 설계 변경판을 구별하는 표시입니다.

오늘 배운 내용 확인하기

평가 · 전자회로·ESP32

이 평가는 전기의 기본량을 말로 설명하고, 센서 기록을 서로 다른 상태로 구분하며, 다음 작업자에게 확인 근거를 전달할 수 있는지 확인합니다. 실제 보드 보유 여부와 코드 입력 속도는 평가하지 않습니다.

준비와 제출 방법

예제 ZIP을 푼 폴더에서 활동별 [work/A번호_worksheet.md](#)를 찾으세요. 앞선 전기량 계산·배선 검토·센서 검수 기록을 모으고, [work/A17_worksheet.md](#)에 종합 결과를, [work/A18_worksheet.md](#)에 인수인계 설명을 적습니다. 인수인계는 다음 사람이 같은 조건으로 작업을 이어 갈 수 있도록 설명과 파일을 넘기는 일입니다.

직접 실행했는지, 종이로 계산했는지, 제공 출력 또는 강사 시연과 비교했는지 표시하세요. 실제 통전이나 측정을 하지 않았다면 “미확인”으로 적습니다.

수행 과제

- 전기량 설명:** 전압·전류·저항의 뜻과 단위 $V \cdot A \cdot \Omega$ 를 연결하세요. 계산식에 숫자와 단위를 함께 적고, GND가 회로에서 어떤 기준점인지와 전류가 흐를 닫힌 경로를 설명하세요.
- 장비 확인표:** 실제 보드·센서 모델, 공급 전압, 사용할 핀, 연결 근거를 적으세요. 자료가 없는 칸에는 추측값 대신 “미확인 / 확인할 자료”를 씁니다. 확인되지 않은 장비를 직접 연결할 필요는 없습니다.
- 모의 센서 검수:** 숫자 0.0, 미측정 null, 숫자로 보이는 문자열, 경계값을 각각 확인하세요. 입력 값·자료형·예상 상태·실제 상태·판단 이유를 한 줄씩 기록합니다. 원시값을 물리량으로 바꾸는 기준이 있는지도 구분하세요.
- 인수인계:** 사용한 입력 파일, 적용 규칙, 결과, 아직 확인하지 못한 실제 장비 조건을 묶어 전달하세요. 모의 코드의 성공과 실제 보드의 성공을 별도로 표시하세요.

최종 제출물은 **전기량 계산과 배선 검토표 + A17 예상·결과표 + A18 인수인계 기록**입니다. 입력·설정·출력 파일이 있으면 함께 첨부하세요.

채점 기준 · 100점

관점	배점	충분한 근거의 기준
전기 개념	25	$V \cdot A \cdot \Omega$ 와 기준점·닫힌 경로를 연결하여 설명하고 계산에 단위를 쓴다.
하드웨어 판단	25	모델·전압·핀맵의 근거를 적으며, 근거가 없을 때 확인 전 보류로 표시한다.
데이터 검수	30	0·null·문자·경계값을 구별하고, 입력과 규칙으로 상태 판단을 설명한다.
재현과 인수인계	20	입력·규칙·결과·미확인 사항을 함께 남겨 다음 사람이 같은 검사를 할 수 있게 한다.

각 관점은 **근거 없음 0% / 일부는 맞지만 핵심 누락 50% / 근거와 결과·범위가 일치 100%**로 채점합니다. 25점 항목은 0·12.5·25점, 30점 항목은 0·15·30점, 20점 항목은 0·10·20점입니다. 총점 70점 이상이며, (1) 모의/실물 검증 범위를 구분하고 (2) 모델·전압·핀 근거가 없을 때 임의로 확정하지 않고 **확인 전 보류**로 표시해야 기본 목표에 도달합니다. 필수 조건이 미충족이면 총점과 별도로 보완 필요를 기록하고, 장비 검토표의 미확인 항목·확인할 자료를 고친 뒤 이유를 다시 설명합니다.

설명으로 확인하는 질문

- 저항 양단에 3.3V가 걸리고 저항이 660Ω이라면 전류는 얼마인가요? A와 mA 단위를 구별해 설명하세요.
- ADC 원시값 하나만 보고 온도라고 말할 수 있나요? 해석에 필요한 추가 정보를 적으세요.
- null과 0.0을 같은 값으로 처리하면 어떤 정보를 잃게 되나요? 센서 측정 상황으로 설명하세요.
- 파일을 사용한 모의 예제가 통과한 뒤, 실제 보드를 연결하기 전에 더 확인할 사항 세 가지는 무엇인가요?

제출 전 확인

- 계산의 숫자와 단위를 함께 썼는가?

- 장비 정보의 확인 근거와 미확인 항목이 나뉘어 있는가?
- 미측정·잘못된 입력·정상값을 구분했는가?
- 실제 실행 여부와 실제 장비 미검증 범위를 밝혔는가?

잘못된 답은 판단에 사용한 입력과 규칙부터 다시 확인합니다. 설명을 수정했다면 바뀐 이유를 인수인계 기록에 한 문장 추가하세요. 정답과 채점 예시는 강사용 자료에 있습니다.

새 상황으로 옮겨 보는 전이 문제 · 복습·도전

교시당 한 문제이며 기존 필수 평가 후 필요에 따라 선택합니다. 독립 작성 또는 판단을 먼저 기록한 뒤 해설을 확인합니다. 설치 실패나 장비 부재는 개념 오답과 구분합니다.

전이 1 · 1교시

저항 양단 2.4V·1200Ω일 때 A와 mA를 계산하고 닫힌 경로를 설명하세요.

정답과 해설

0.002A=2mA입니다. 전원 +에서 저항을 거쳐 -로 돌아오는 길을 포함합니다.

전이 2 · 2교시

공급 5V·가정 LED 강하 2V·저항 1000Ω의 계산 전류와 GPIO 연결 승인 여부를 쓰세요.

정답과 해설

3mA입니다. 수학 계산만으로 연결을 승인하지 않으며 정확한 보드·핀·부품 정격을 확인해야 합니다.

전이 3 · 3교시

이상적 03.3V·04095 끝점 대응에서 코드 4095와 JSON null의 의미를 각각 쓰세요.

정답과 해설

4095는 모델의 3.3V, null은 값 없음입니다. 실물 교정이나 섭씨 온도를 직접 뜻하지 않습니다.

전이 4 · 4교시

초기 True, 문턱 28/27.5, 입력 [27.6,27.5,None,28]의 상태를 추적하세요.

정답과 해설

True,False,False 유지와 no command,True입니다. 결측을 영도로 바꾸지 않고 상태와 측정 유효성을 나눕니다.

전이 5 · 5교시

Serial.println으로 고정 25를 출력했습니다. 센서와 직렬 전송 검증의 상태를 나눠 쓰세요.

정답과 해설

실제로 보드·포트에서 관찰했다면 고정 문자열 전송만 확인한 것입니다. 센서 측정 정확도는 그 코드에 읽기 단계가 없어 미검증입니다.

전이 6 · 6교시

경계가 28에서 30으로 바뀌었습니다. 새 시험 세 입력과 실물 미확인 항목을 쓰세요.

정답과 해설

29.9,30.0,30.1 및 결측을 시험하고 기대 상태를 갱신합니다. 보드 신호 전압·센서 오차 등은 별도 실물 확인입니다.

파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

전자회로·ESP32 PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.